

10.3.3 Nav2 导航系统与路径规划

课程信息

- 环境：Windows 10、WSL2 Ubuntu 24.04、ROS 2 Jazzy、Gazebo Harmonic、**Nav2** (Navigation 2)。
- **认知主线**（先于组件名词）：补清第二节的漂移与 `map → odom` → 加载冻结地图 → 初始定位 → 行走中矫正漂移 → 规划跟线到目标。
- 动手目标：单目标 / 命名目标 / 巡航与取消；**CP3** (Checkpoint 3) 验收。
- 主路径：Gazebo + RViz；兜底：lite + 无界面 CP3。

路径约定：本讲义假设项目位于 `~/mecamind_ros2`。若路径不同，请自行替换。建议先 `git fetch --tags` 且 `git checkout lesson3.0`（第三节课 tag 发布后），或使用课程指定的当前分支。

本节不要求 YOLO、跟随或语音；后续课程会复用本节的导航与任务接口。

漂移的公式与实测曲线见配套专题：`docs/10.3.3_专题_里程计漂移与矫正.md`。

先读这段：本节主线（比背组件名更重要）

第二节课做出了地图，也留下了一条「奇怪」的 TF：`map → odom`。当时多数人只记住「有这条边」，**还没真正讲清：轮子为什么会漂、为什么不能直接改里程计、矫正量从哪来。** 第三节必须先把这件事补完，再谈导航——否则后面一堆 Nav2 节点名会变成名词堆砌。

导航本身只回答四个问题，按时间顺序就是：

- ① 地图怎么进系统？ → 加载冻结的 PGM/YAML（不再边走边建）
- ② 一开始我在地图哪？ → 给初始位姿 + 激光对照地图（AMCL）
- ③ 走起来又漂了怎么办？ → 仍用 `map→odom` 记账；发布者从 SLAM 换成 AMCL
- ④ 怎么开到目标点？ → 在代价地图上规划 + 局部跟线（这才轮到 planner/controller）

读下文时请按这个顺序；节点名只是实现这四问的工具。更细的漂移推导与实测曲线见配套专题《里程计漂移与矫正》。

1. 本节学习目标

读完并做完 LAB 后，应当能够：

1. 用自己的话讲清第二节没讲透的点：里程计会漂；`map → odom` 是矫正账本，不是第三个「世界」。
2. 说清导航主线四步：加载地图 → 初始定位 → 行走中矫正漂移 → 规划并跟线到目标。
3. 解释为何一开始要给初始位姿，以及行走中 AMCL 如何、为何能把漂移补回来。

4. 用四个比喻讲清「怎么去」的四层：代价地图（危险度地形）、全局规划（水漫地形找谷底）、DWB（每 0.1 秒预演几百种未来）、行为树（领班按预案调度）。
5. 用 `navigation.launch.py` 跑通单目标 / 命名目标，并用 `scripts/test_cp3.sh` 完成 CP3。

（组件全称、工程里新增了哪些文件，集中放在文末 **§ 2.7 速查**，需要时再翻。）

2. 主线详解

2.1 先补第二节：漂移是什么，`map` → `odom` 是干什么的

两类「我在哪」

来源	做法	本项目载体	特点
航迹推算	数轮子转了多少圈，积分出位置	<code>/odom</code> 、TF <code>odom</code> → <code>base_footprint</code>	快、连续， 越走越偏
环境观测	激光「照镜子」，和地图比对	<code>/scan</code> + 地图	较准，但慢，修正时可能跳一下

漂移来自积分：每步误差很小，但只进不出。麦轮横移、原地转时打滑更明显；朝向偏 1°，直行 5 m 大约横向偏 9 cm。纯信轮子走一圈，墙会对不齐。

为什么不直接改 `/odom`

最直觉的修法是：激光算准了，把里程计改成真值。这样控制环会看见「车瞬移」，速度会抖。ROS 的做法是拆成两段：



- 控制器只看连续的 `odom` → `base`。
- 规划 / 任务看完整的 `map` → `base` = 两段相乘。
- `map` → `odom` 不是传感器读数，是「真实估计 - 里程计」算出来的差。

类比：闭眼数步子不改步数；偶尔睁眼看路牌，只把「起点」整体挪一下。

第二节建图时，这个差由 `slam_toolbox`（扫描匹配）发布；第三节地图冻住了，改由 `amcl`（粒子滤波）发布——同一条边，换算法。

公式、数值例子、Gazebo 实测阶梯曲线（修正约几厘米到十几厘米）：见 `docs/10.3.3_专题_里程计漂移与矫正.md`。

记住一句即可：里程计管连续，激光管正确，`map` → `odom` 是差值账本；不为零是正常，恒为零反而要怀疑定位没在工作。

2.2 主线①：地图怎么加载进导航？

第二节产物是磁盘上的 PGM（格子图）+ YAML（分辨率、原点等元数据）。
第三节不再跑 SLAM 画图，而是：

- 1. map_server 读入 YAML，发布话题 /map（占用栅格，和建图时消息类型相同）。
- 2. 默认加载包内合格图 mecamind_three_room_nav.*；也可用自己的图：

```
ros2 launch mecamind_bringup navigation.launch.py \
  map_file:=$HOME/mecamind_ros2/maps/你的地图.yaml
```

- 3. 规划与定位都消费这份冻结地图——商场平面图已经印好，不再边逛边重画。

	第二节（建图）	第三节（导航）
谁提供 /map	slam_toolbox 实时生成	map_server 读文件
地图会变吗	会随探索变大变清晰	冻结，除非你换文件重启

不要同时开 mapping.launch.py 和 navigation.launch.py：两个发布者会抢 map → odom。

2.3 主线②：一开始怎么知道自己在地图的哪个位置？

地图加载后，机器人仍可能「站在原点却以为在卧室」——定位需要初值。

本项目启动约 8 秒后，lifecycle_activator 会：

- 1. 按序激活 Nav2 生命周期节点（先有地图，再定位，再规划控制）；
- 2. 向 /initialpose 发布初始位姿（默认地图原点，与仿真出生点对齐）。

AMCL（Adaptive Monte Carlo Localization，自适应蒙特卡洛定位）在此初值附近撒一群「猜测」（粒子），用激光对照 /map 给猜测打分，留下说得通的，收成一小团——这就是「我在哪」。

为什么必须给初值？粒子滤波是局部收敛的：初值错了房间，激光怎么比对都不像，猜测收不拢。RViz 里车「瞬移」、路径乱飘时，用 2D Pose Estimate 在真实位置点一下，等于重新告诉它从哪家店门口开始猜。

商场类比：睁眼后先告诉系统「大概在一楼中庭」，再靠招牌把猜测收紧；不能从「我在三楼」硬对一楼的墙。

自检：

```
ros2 topic echo /amcl_pose --once          # 位姿应接近初值附近
ros2 run tf2_ros tf2_echo map base_footprint --ros-args -p use_sim_time:=true
```

2.4 主线③：走起来漂移怎么解决？为什么能解决？

车一动，轮子继续积分 → odom → base 继续漂。

每隔一小段路（本项目大约走 0.15 m 或转 0.15 rad），AMCL 再拿当前激光去对静态地图：

1. 估出较准的 ($T_{\{\mathrm{map}\}\rightarrow\{\mathrm{base}\}}$)（粒子云加权均值）；
2. 用它和当前里程计做差，更新 ($T_{\{\mathrm{map}\}\rightarrow\{\mathrm{odom}\}}$)；
3. 里程计本身不改写，只改账本。

为什么这样就能解决？

- 室内墙和家具提供稳定「路牌」；激光端点应对齐墙上 → 估真实位姿有依据。
- 短期仍信里程计（平滑、高频）；激光只做低频结算 → 控制不抖。
- 地图已冻结且与建图起点对齐 → 「照镜子」有固定参照。

为什么有时解决不了？初值错房间、长走廊特征太少、运动模型选错（麦轮必须用 Omni 全向模型，差速模型会把横移当噪声）→ 粒子锁到错误假设，只能重新给初值。

RViz 勾选 **Particle Cloud**：静止时较散，走几米后收紧 = 矫正在生效。

想直接看账本：

```
ros2 run tf2_ros tf2_echo map odom --ros-args -p use_sim_time:=true
```

运动后 Translation 呈**阶梯状**跳变是正常的（不是 bug）。

2.5 主线④：知道自己在哪之后，怎么走到目标？

定位解决「我在哪」；这一步解决「怎么去」。它不是一个算法，而是一条**三层流水线**加一个**调度**——用开车打比方：

代价地图	=	把世界改绘成「危险度地形图」	（地图上先标出哪里不能贴）
全局规划	=	出发前在图上定一条路线 /plan	（去哪条路 —— 秒级）
局部控制	=	每 0.1 秒决定这一脚油门方向盘	（DWB → /cmd_vel —— 100 毫秒级）
行为树	=	领队：正常就"规划→跟线"，出事按预案自救、重规划	

注意快慢分层：路线可以慢慢算、偶尔重算；油门必须高频、连续。这和 § 2.1 「激光管低频正确、里程计管高频连续」是同一种设计哲学。

2.5.1 代价地图：先把地图改绘成「危险度地形」

规划器**不直接**在第二节那张黑白格子图上找路。原因很物理：机器人不是一个点，是一块约 0.40 m × 0.30 m 的车身（参数里的 footprint 四个角点）。一个紧贴墙的格子在黑白图上是「可通行」，车身开进去就是蹭墙。

Nav2 的做法是把栅格图加工成**代价地图 (costmap)**：

1. 障碍格子本体标为**致命**（青色粗边，绝对禁入）；

2. 从每个障碍向外铺一圈**指数衰减**的「代价坡」：离墙 0 m 代价最高，往外按 `cost_scaling_factor: 3.0` 衰减，到 `inflation_radius: 0.45` m 处归零（RViz 里的紫红渐变圈）；
3. 效果：地图从「能走 / 不能走」的**黑白判断**，变成「这里危险 8 分、那里危险 2 分」的**连续地形**。后面所有算法都在这张地形上「捡低处走」，不需要再写任何「离墙远一点」的专门规则。

类比：把每面墙都想成刚刚完的油漆——挨得越近越容易蹭一身。代价地图就是提前把「油漆味浓度」画在图上，找路的人自然会沿味道淡的地方走。

系统里同时维护**两张**代价地图，分工和挂靠的坐标系都不同：

	全局代价地图	局部代价地图
覆盖范围	整张冻结地图	机器人周围 3×3 m 滚动窗口
数据来源	静态地图层 + 实时激光层 + 膨胀层	仅实时激光 + 膨胀层
挂在哪个坐标系	map 系（准确）	odom 系（连续）
供谁消费	全局规划器 planner	局部控制器 DWB

局部挂 `odom` 系正是 § 2.1 二段式设计的直接受益者：AMCL 结算漂移时 `map→odom` 会跳一格，规划器（慢速决策者）无所谓，但 10 Hz 的控制回路受不了脚下的地图突然挪动——所以脚下这 3×3 m 窗口故意「看不见」跳变。

2.5.2 全局规划：一滴水从目标点漫过地形（NavFn）

本项目的规划器是 NavFn，内核是 Dijkstra **波前扩散**。不用记公式，记这个图像：

1. 在**目标点**倒一滴水，让它向四周漫开；代价越高的格子，水漫过去越「费劲」，致命格子漫不过去。
2. 水漫到机器人脚下就停。此时每个被漫过的格子都记着一个数：「从目标漫到我这，总共费了多少劲」。
3. 机器人从脚下出发，每步都往「费劲值下降最快」的邻格走——**逆流而上回到源头**，走出来的这串格子就是全局路径 `/plan`。

这个图像能一次解释 LAB 里将看到的三个现象：

- **路径不走直线**：直线要翻膨胀坡，「总费劲」反而比绕一点远路更大——planner 找的是**代价最低**，不是距离最短。
- **路径始终离墙约一个膨胀半径**：代价坡把路径「挤」到谷底，这就是 0.45 m 安全距离的由来。
- **目标点被占住也能落点**：`tolerance: 0.5` 允许在目标 0.5 m 范围内就近找可达点，不至于整单失败。

格子搜索出来的路径是 45°/90° 的锯齿折线，`smoother_server` 会再把它磨圆一些，机器人跟起来才不别扭。

2.5.3 局部控制 DWB：每 0.1 秒预演 960 种未来

有了路线还要有人踩油门。DWB（Dynamic Window Based）每个控制周期（本项目 10 Hz）做一场「闪电预演」，五步：

1. **开窗**：以当前速度为中心，按加速度限幅（0.8 m/s²）算出下个周期**物理上够得着**的速度范围——这就是「动态窗口」四个字的含义：车不能从 0 瞬间跳到 0.35 m/s，能选的速度永远只有当前速度附近一小

窗。

2. **采样**：在窗口里撒候选速度： v_x 10 档 $\times$ v_y 6 档 $\times$ ω 16 档 = **960 条**。注意 v_y 有 6 档不是 0 档——差速车横向速度恒为零，麦轮可以横移，「斜着走」「平移进门」都是合法候选。这就是参数里 `min_vel_y/max_vel_y:  $\pm 0.25$` 的意义。
3. **预演**：每条候选速度向前模拟 `sim_time: 1.7` 秒，得到一条假想轨迹画在局部代价地图上。
4. **打分**：一排 critic「评委」各打各的分，加权求和：`PathAlign/PathDist`（贴住全局路径，权重 32）、`GoalAlign/GoalDist`（朝向目标，权重 24）、`BaseObstacle`（压到高代价区扣分）、`Oscillation`（防止来回抖）、`RotateToGoal`（到点后负责原地转向对齐）。权重决定「话语权」：本项目贴线评委权重最高，所以车总体老实沿线走，遇障碍才临时偏离。
5. **执行**：取总分最高那条轨迹的**第一步速度**发出去；0.1 秒后一切推倒重来。

类比：老司机开车不是背着地图走，而是每一瞬间在脑内预演「方向盘打这个角度会怎样」，从几百个「怎样」里挑一个既不偏航又不刮蹭的。DWB 就是把这个直觉写成了循环。

选出的速度并不直接到轮子，还要过两道关（§ 2.6 速度链）：`velocity_smoother` 限加速度防急启急停，`safety_gate` 拿激光做最后一道急刹。

2.5.4 行为树 `bt_navigator`：一切顺利之外的剧本

规划和控制都可能失败：新障碍把路堵死、门口卡住不动、打滑原地空转。`bt_navigator`（BT = Behavior Tree，行为树）把「正常流程 + 全部预案」编排成一棵树：

- **正常循环**：约 1 秒重规划一次全局路径 + DWB 持续跟线——这就是 LAB 里「路径起点始终长在机器人脚下」的原因。
- **判卡住**：`progress_checker` 发现 12 秒内没挪动 0.35 m，判定「卡住」。
- **自救序列**（`behavior_server` 提供）：`spin` 原地旋转——让激光扫一整圈刷新代价地图，把已经移走却还残留在图上的「幻影障碍」清掉；`backup` 倒车脱困；`wait` 原地等待（等行人走开）。每做完一步就重新规划试一次。
- **全部失败**才向上报 `aborted`，由任务层决定重试还是放弃。

类比：行为树是餐厅领班——厨师（`planner`）出菜、服务员（DWB）端菜，客人打翻了盘子按预案处理（收拾、重做、换桌），实在处理不了才上报经理（任务层）。领班自己不做菜也不端盘子，只管「现在该谁上、失败了走哪条预案」。

人说「去卧室」时，中间还有一层任务层：查命名点表 $\rightarrow$ 调 `NavigateToPose` action $\rightarrow$ `/mecamind/mission_state` 心跳。这是主线之上的「遥控器」，LAB 2/3 再练。

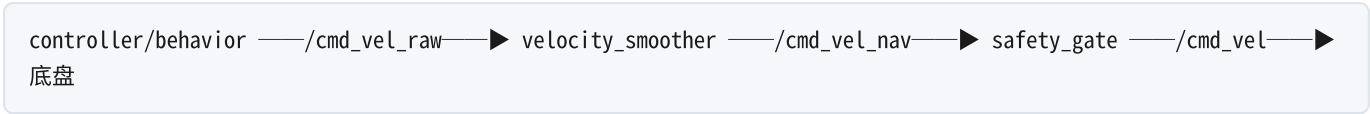
到这里，第三节的**认知主线**就闭环了：

冻结地图 $\rightarrow$ 初值定位 $\rightarrow$ 行走中用激光结算漂移 $\rightarrow$ 规划并跟线到目标

下面 § 2.6~2.7 是支撑主线的工程细节与名词表；LAB 从动手验证主线开始。

2.6 支撑细节：速度链、命名目标、三种刹车

速度链（排「车不动」时按跳查）：



相对第二节，多了 raw → smoother；门与最终 /cmd_vel 仍对齐双后端。

命名目标： /mecamind/task_command JSON → mission_executor 查 mecamind_named_goals.yaml → NavigateToPose action。

话题点目标适合 RViz；程序化取消必须用 action（可撤销的「订单」）。

三种停：任务 stop（取消订单）/ 激光门（保险杠）/ 命令超时（遥控器断流）。互相独立。取消要在运动中做才有意义。

麦轮参数要点：AMCL 用 OmniMotionModel；DWB 打开 vy（可横移）。

2.7 速查：新名词、本课新增模块、节点图

新名词（相对第 1、2 课）

缩写 / 名	全称	在主线里出现的位置
Nav2	Navigation 2	整套导航框架
AMCL	Adaptive Monte Carlo Localization	主线②③：已知地图定位 + 发 map→odom
BT / bt_navigator	Behavior Tree	主线④：编排规划/跟踪/恢复（§ 2.5.4）
DWB	Dynamic Window Based	主线④：局部预演选速度（§ 2.5.3）
costmap	Cost Map	主线④：危险度地形图（§ 2.5.1）
NavFn	Navigation Function (Dijkstra 波前)	主线④：全局搜路径（§ 2.5.2）
inflation	Inflation Layer（膨胀层）	主线④：障碍外的代价坡，半径 0.45 m
map_server	Map Server	主线①：加载冻结地图
action / NavigateToPose	ROS 2 Action	任务层下单
CP3	Checkpoint 3	本节验收
RTF	Real-Time Factor	仿真快慢

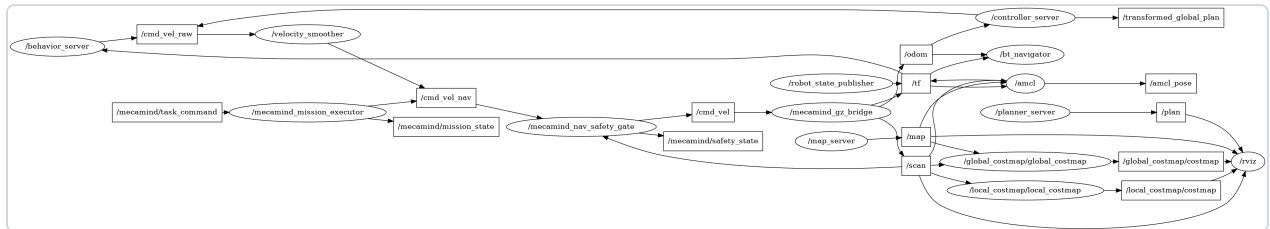
本课新增 vs 复用

新启用： navigation.launch.py；Nav2 节点（map_server、amcl、planner、controller、behavior、bt_navigator、velocity_smoother 等）；mecamind_nav2_params.yaml；命名点/巡航 yaml；mission_executor；CP3。

复用但改角色：双后端接口不变；SLAM 停用；map→odom 改由 amcl 发；safety_gate 仍在末端；lifecycle 改为激活 Nav2 并发 /initialpose。

不要同时开：mapping.launch.py / slam_toolbox。

运行时节点图（可选精读）



导航栈运行时节点与话题图

三条线索对应主线：

1. **定位链：** /map + /scan → amcl → /amcl_pose 与 TF 中的 map→odom（主线②③）。
2. **速度链：** raw → nav → cmd_vel → 桥（主线④的执行出口）。
3. **任务链：** task_command → mission_executor → mission_state；到 bt_navigator 走 action，图上常看不到普通话题线。

排错时按主线问：地图加载了吗？初值对吗？map→odom 在跳变吗？速度卡在哪一跳？

3. 启动前准备

```
cd ~/mecamind_ros2
bash scripts/cleanup_ros2.sh --all      # 清掉上次课残留的 gz/rviz/slam 进程
source /opt/ros/jazzy/setup.bash
source install/setup.bash
export DISPLAY=:0                        # WSLg 图形输出
```

确认第二节课的地图资产在（本节默认用打包进 mecamind_tools 的 CP2 地图，无需手动指定）：

```
ls $(ros2 pkg prefix mecamind_tools)/share/mecamind_tools/maps/
# 应看到 mecamind_three_room_nav.pgm / mecamind_three_room_nav.yaml
```

如果你想用自己第二节课建的地图：

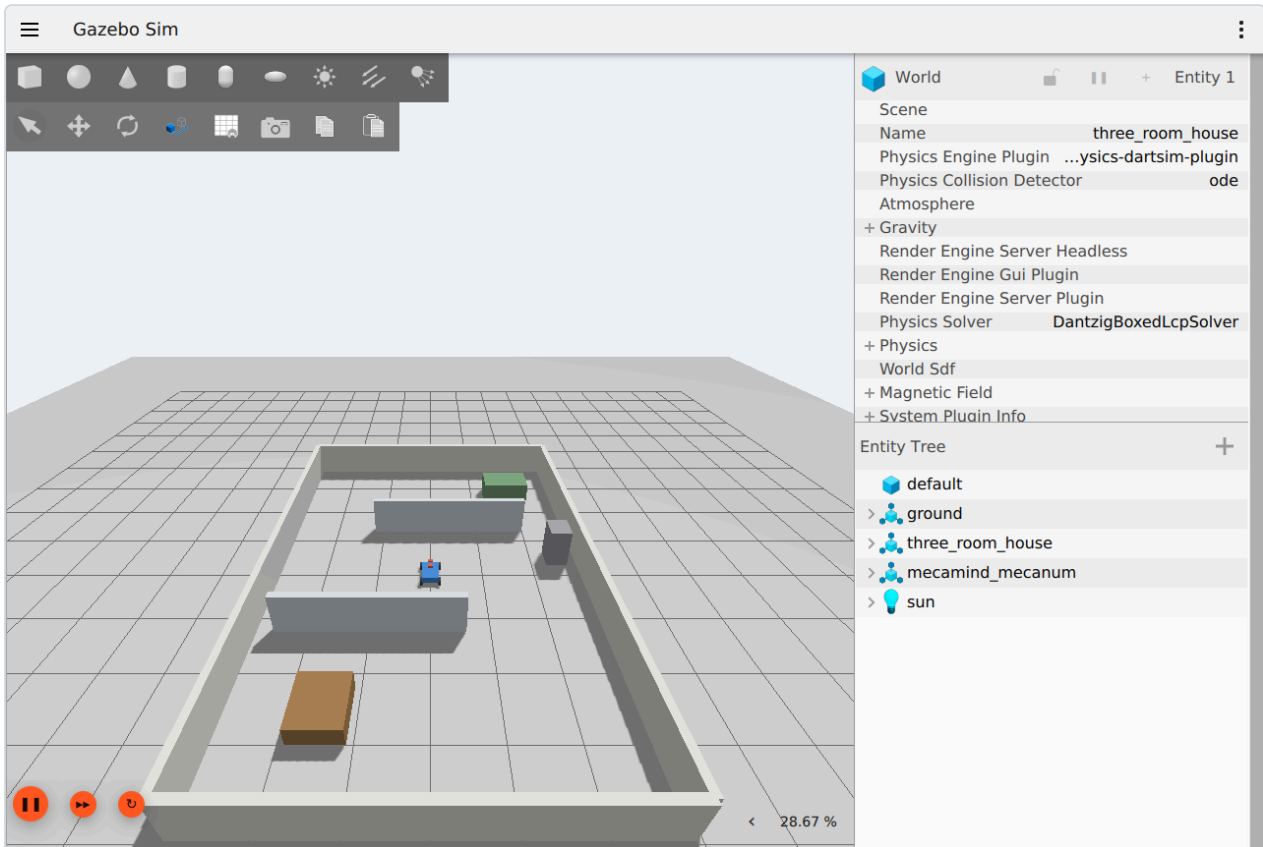
```
ros2 launch mecamind_bringup navigation.launch.py map_file:=$HOME/mecamind_ros2/maps/mecamind_cp2_gazebo_map.yaml
```


4. LAB 1: 启动导航栈并认识界面 (Gazebo + RViz)

4.1 启动

```
ros2 launch mecamind_bringup navigation.launch.py
# 等价于 backend:=gazebo use_gui:=true with_rviz:=true
```

Gazebo 窗口里是熟悉的三居室场景，机器人出生在地图原点（与第二节课建图起点一致）：



Gazebo 导航场景

启动后 8 秒左右，`lifecycle_activator` 会自动完成：

1. 依次 `configure` + `activate` 九个 Nav2 生命周期节点（日志里能看到逐个 `transition succeeded`）。
2. 向 `/initialpose` 发布初始位姿（地图原点，与机器人出生点一致）。

生命周期可以想成：仪器先接通电源并自检（`configure`），再拨到“开始工作”（`activate`）。Nav2 核心节点默认不会一启动就干活，要等 `activator` 按顺序按按钮——所以前几秒发目标常会失败，不是你命令写错了，是仪器还没拨到工作档。

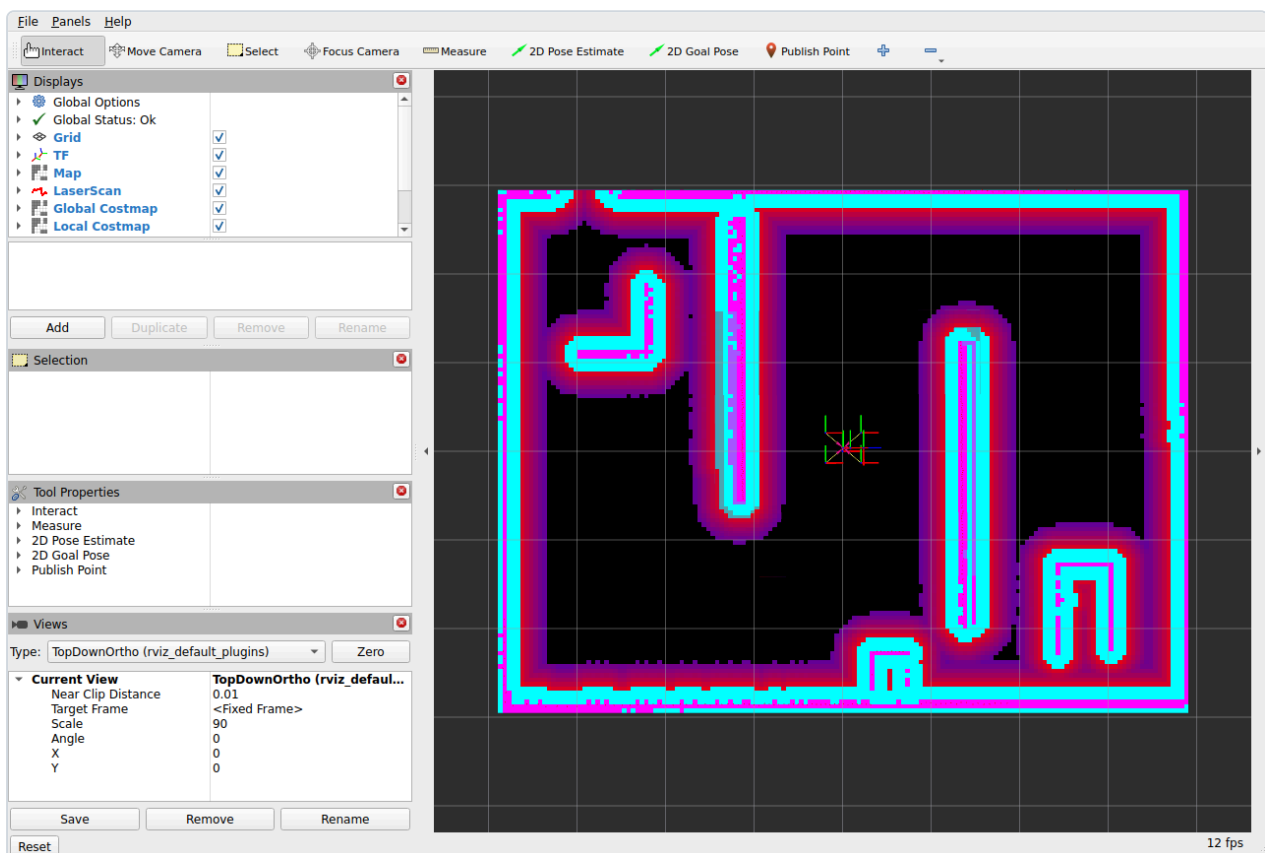
4.2 检查清单（3 分钟）

```
# 定位是否收敛：应看到 x/y 接近 0, covariance 逐渐变小
ros2 topic echo /amcl_pose --once

# TF 链是否完整：map → odom → base_footprint
ros2 run tf2_ros tf2_echo map base_footprint --ros-args -p use_sim_time:=true

# 九个生命周期节点是否都 active
ros2 lifecycle get /bt_navigator
```

RViz 里应看到：灰白静态地图、彩色代价地图（障碍物周围的膨胀圈）、红色激光点贴在墙上、TF 坐标轴在地图原点。下面是启动完成后的实际画面（俯视正交视角）：



RViz 导航栈启动完成

读图要点：

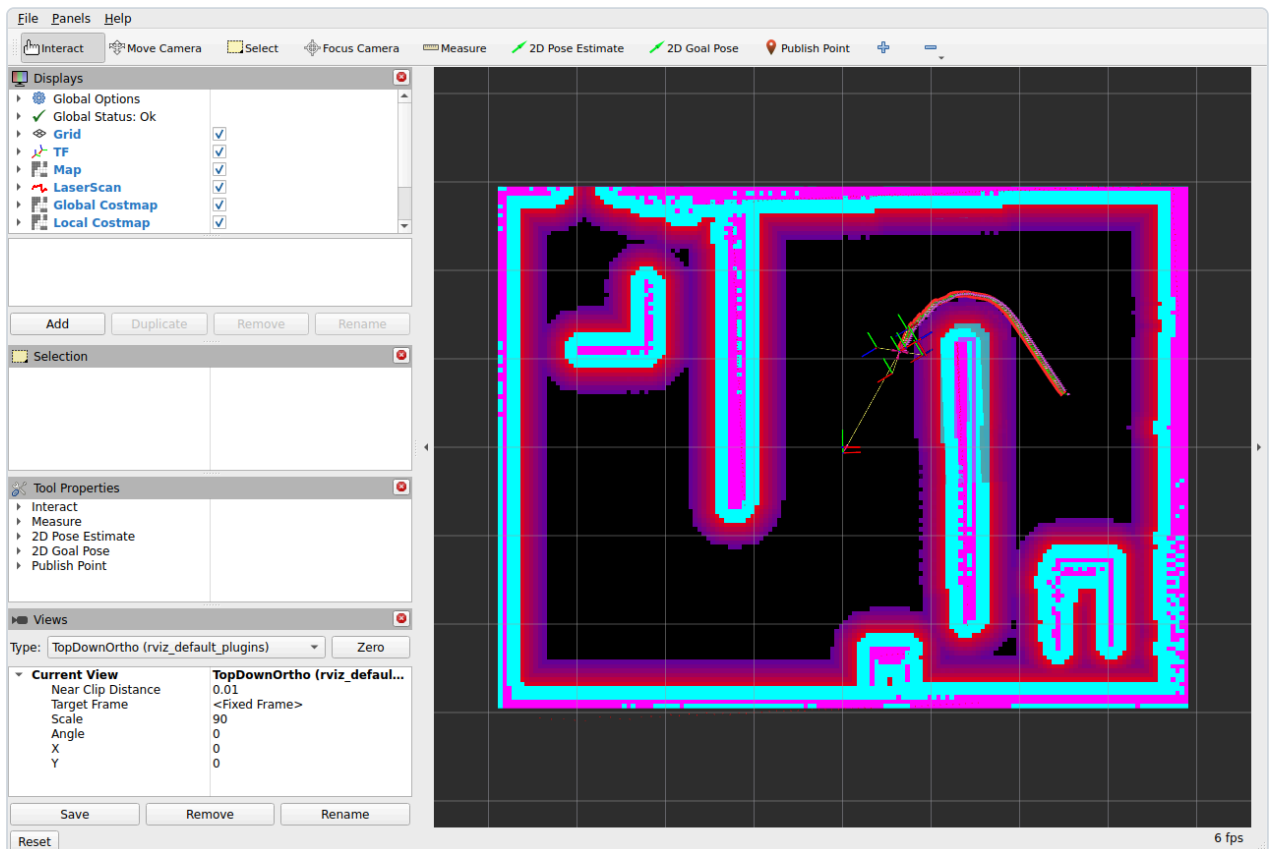
- **青色粗边**：代价地图的障碍物本体（激光实时命中的墙和家具）。
- **紫红色渐变圈**：膨胀层——离障碍越近代价越高，规划器会自动绕开；这圈的宽度就是 `inflation_radius`: 0.45。
- **中央坐标轴**：机器人 TF (`base_footprint` 及各轮系)，此刻在地图原点，说明 `initialpose` 已生效。
- 左侧 Displays 里 Map、LaserScan、Global/Local Costmap 全部打勾且 Global Status 为 Ok，是“启动健康”的最快判据。

4.3 单目标导航

在 RViz 顶部工具栏点 **Nav2 Goal** (或 2D Goal Pose)，在客厅位置按下并拖动指定朝向，松开：

- 红色曲线：planner 输出的全局路径 `/plan`。
- 机器人开始沿路径移动，局部代价地图跟着机器人滚动。
- 到点后自转对齐目标朝向。

下图是发出"去卧室"目标后的实际画面——红色路径从机器人当前位置绕过中间隔墙、翻过墙角伸进卧室：



导航中的全局路径

读图要点：

- 路径**不走直线**：直线会穿过隔墙的膨胀区，planner 在代价地图上找的是"代价最低"而非"距离最短"的路。
- 路径末端悬停在卧室内的目标点；机器人移动过程中 planner 默认约 1 秒重算一次，路径起点始终"长"在机器人脚下。
- 机器人周围一小圈更亮的区域是**局部代价地图**的滚动窗口，controller 只看这一圈做避障。

命令行等价操作（用来理解：RViz 只是图形前端，背后仍是同一个 action）：

```
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose \
  "{pose: {header: {frame_id: map}, pose: {position: {x: 0.0, y: 0.4}, orientation: {w: 1.0}}}}"
```

4.3.1 边导航边验证原理（把 § 2.5 的四层各抓一次现行）

机器人在路上时，另开终端逐条运行——每条命令对应主线④的一层：

```
cd ~/mecamind_ros2 && source install/setup.bash

# ① 全局规划层: /plan 约 1 秒重发一次，且第一个点始终在机器人脚下
# (行为树在周期性重规划——路径是"活"的，不是发一次就完)
ros2 topic hz /plan

# ② 局部控制层: DWB 每 0.1 秒预演 960 条轨迹后发出的"这一脚油门"
# 直行时看 linear.x; 斜穿房间时能看到 linear.y 非零——差速车永远做不到
ros2 topic echo /cmd_vel_raw

# ③ 速度链: raw(10Hz 决策) → nav(平滑限加速度) → cmd_vel(过安全门)
# 三跳频率与数值对比一遍，"车不动了查哪一跳"就有了体感
ros2 topic hz /cmd_vel_raw /cmd_vel_nav /cmd_vel

# ④ 定位层: 行走中 map→odom 仍在阶梯状结算漂移 (主线③没有下班)
ros2 run tf2_ros tf2_echo map odom --ros-args -p use_sim_time:=true
```

再做一个眼见为实的对照实验：在 RViz 里给**同一个目标点**从不同起点各发一次，观察路径每次都不同但**永远沿膨胀圈的"谷底"走**——这就是 § 2.5.2 「水漫地形、逆流回源」的宏观表现；路径与墙的间距目测约 0.45 m，正是 `inflation_radius` 的值。

4.4 观察 AMCL 粒子云

RViz 左侧 Displays 里勾选 **Particle Cloud**。机器人静止时粒子云较散；移动几米后明显收缩——这就是"运动 + 激光观测让定位收敛"的直观证据。

5. LAB 2: 命名目标导航

保持 LAB 1 的导航栈运行，另开一个终端：

```
cd ~/mecamind_ros2 && source install/setup.bash

# 盯着任务状态 (保持这个窗口可见)
ros2 topic echo /mecamind/mission_state
```

再开一个终端发任务：

```
cd ~/mecamind_ros2 && source install/setup.bash

# 去客厅
ros2 topic pub --once /mecamind/task_command std_msgs/msg/String \
  '{data: "{\"intent\": \"navigate\", \"target\": \"living_room\"}}'

# 去卧室 (支持中文别名)
ros2 topic pub --once /mecamind/task_command std_msgs/msg/String \
  '{data: "{\"intent\": \"navigate\", \"target\": \"卧室\"}}'
```

mission_state 窗口会依次出现：

```
queued:living_room → sending:living_room → active:living_room → reached:living_room
```

命名点定义在 `src/mecamind_tools/config/mecamind_named_goals.yaml`，可以自己加点：坐标可以在 RViz 里用 **Publish Point** 点地图读出（左下角状态栏显示坐标）。

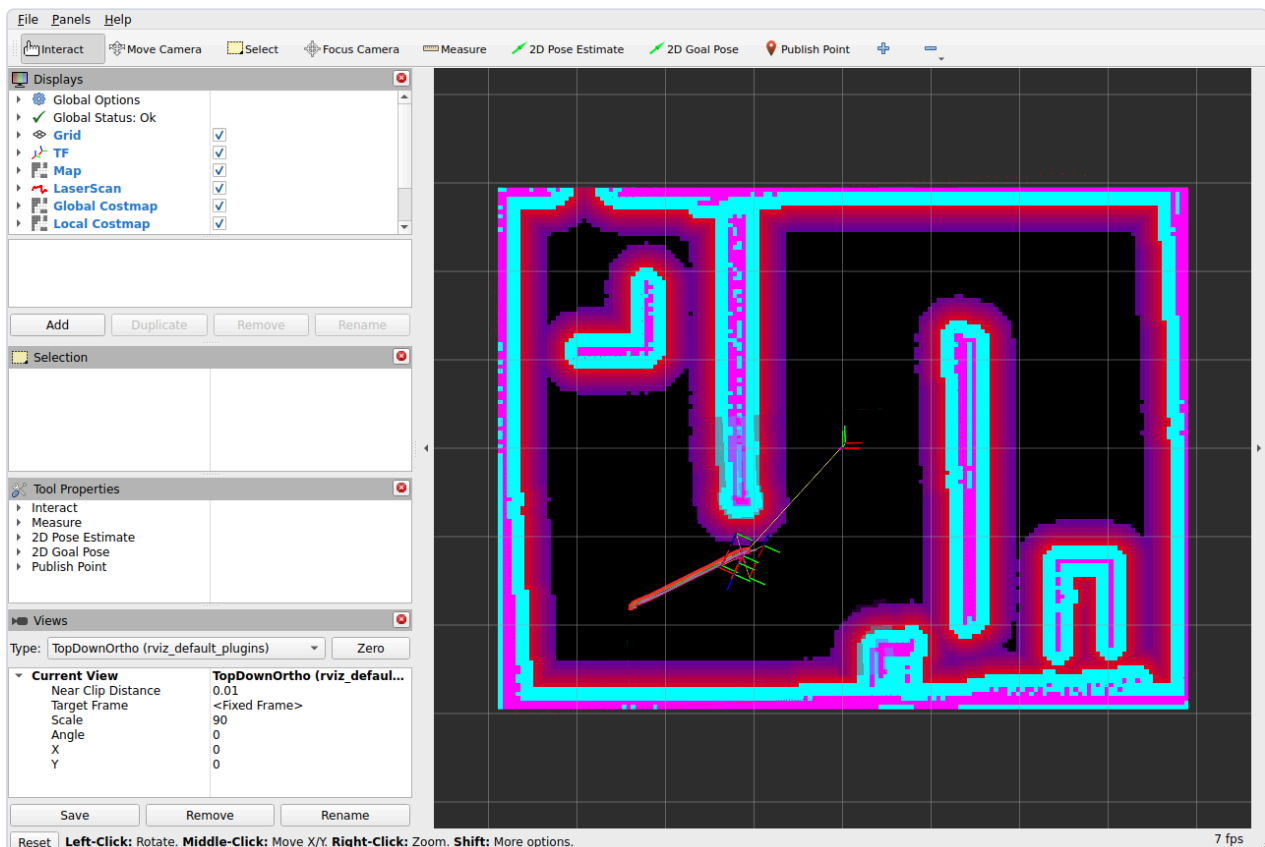
6. LAB 3：多点巡航与取消

6.1 巡航

```
ros2 topic pub --once /mecamind/task_command std_msgs/msg/String \
  '{data: "{\\"intent\\": \\"patrol\\"}"}'
```

机器人按 `mecamind_patrol_waypoints.yaml` 依次走 `hall_entry` → `living_room` → `bedroom`，mission_state 里能看到一个点 `reached` 后自动 `queued` 下一个点，最后 `patrol_complete`。

下图是巡航途中前往 `living_room`（左下方向）的实际画面，红色路径段指向下一个巡航点：



巡航中的机器人与路径

6.2 运动中取消

在机器人正在移动时发送：

```
ros2 topic pub --once /mecamind/task_command std_msgs/msg/String \
  '{data: "{\\"intent\\": \\"stop\\"}"}'
```

预期：机器人立即减速停车，mission_state 变为 mode=idle, detail=stopped，RViz 里路径消失。

6.3 安全门控演示（Gazebo）

导航过程中，在 Gazebo 里把一个障碍物拖到机器人正前方 0.2m 内：

```
# 观察安全门状态：level 变成 stop, reason=front_stop
ros2 topic echo /mecamind/safety_state
```

障碍移开后自动恢复。这是独立于 Nav2 的最后一道保险。

6.4 （可选）行为树自救演示：把路堵死看它怎么办

想亲眼看看 § 2.5.4 的预案生效：发一个导航目标，趁机器人还没到，在 Gazebo 里把一个障碍物拖到门口把路完全堵死，然后盯着 launch 终端的日志：

1. DWB 找不到能走的轨迹，progress_checker 约 12 秒后判「卡住」；
2. 行为树按预案调用恢复行为——日志里出现 spin（原地转一圈刷新代价地图）或 backup（倒车脱困），随后重新规划；
3. 若此时把障碍移开，重规划成功，机器人继续上路；一直堵着则最终 aborted，mission_state 报失败。

对比 § 6.3 可以体会两层「处理障碍」的分工：代价地图 + 重规划是聪明地绕，安全门是来不及了先刹死——前者是导航智能，后者是保险杠，互不替代。

7. LAB 4：轻量仿真导航（兜底）

Gazebo 跑不动的电脑用轻量后端，接口和操作完全一致：

```
ros2 launch mecamind_bringup navigation.launch.py backend:=lite use_gui:=false
```

RViz、任务命令、巡航、取消全部照旧。轻量后端的 RTF 稳定为 1.0，导航速度反而比低配电脑上的 Gazebo 更快。

8. CP3 自动验收

一条命令完成"启动导航栈 → 连续三个命名目标 → 运动中取消"的全流程验收：

```
# 轻量后端（默认，约 3 分钟）
./scripts/test_cp3.sh

# Gazebo 后端（视电脑性能 10~20 分钟）
MECAMIND_CP3_BACKEND=gazebo ./scripts/test_cp3.sh
```

通过标准（未行输出 `MECAMIND_CP3_OK`）：

1. `living_room`、`bedroom`、`hall_entry` 三个命名目标全部 `reached`。
2. 取消测试：目标激活后 3 秒发 `stop`，机器人停止且状态回到 `idle`。
3. 全程报告写入 `maps/mecamind_cp3_<backend>_nav_report.json`。

失败时报告会附带**失败证据**：最后一次 AMCL 位姿与协方差、安全门状态、任务状态历史，便于课后离线定位问题。

实测基准（本讲义编写时在 WSL2 上的真实通过数据，单位：秒，为**仿真时间**）：

目标	lite 后端耗时	Gazebo 后端耗时	备注
living_room	11.1（2 次尝试）	20.6（2 次尝试）	首次尝试撞上 AMCL 刚收敛的窗口，重试机制自动兜住
bedroom	62.5	241.4	跨全图 + 绕两道隔墙，Gazebo 低 RTF 下最耗时
hall_entry	22.7	45.7	—
取消测试	通过	通过	激活后 3s 发 <code>stop</code> ，均立即停车回 <code>idle</code>

两个后端 `overall_ok: true`。耗时参考：lite 全程约 3 分钟，Gazebo 视机器性能约 10~20 分钟——**建议验收优先用 lite**；Gazebo 适合机器性能较好时自测观感。

9. 常见问题

9.1 目标一发就 failed，日志报 "Initial robot pose is not available"

AMCL 还没发布 `map → odom`（初始位姿没给或 TF 还没积累够历史）。等 `lifecycle_activator` 的 `initialpose` 日志出现后再发目标；手动补救：RViz 里用 2D Pose Estimate 在机器人真实位置点一下。

9.2 机器人在地图上"瞬移"或粒子云发散

初始位姿给错了。AMCL 是局部收敛的，猜错房间就再也拉不回来——像坚持"我在三楼"去对一楼的招牌。用 2D Pose Estimate 在正确位置重新初始化（重新告诉它从哪家店门口开始猜）。

9.3 路径贴墙、门口反复卡住

膨胀半径过小（或激光噪声导致门框"虚胖"）。本项目 `inflation_radius: 0.45`；如果你改小了，恢复它。安全门 warn 区（前方 0.45m）也会主动降速，门口慢是正常现象。

9.4 Gazebo 后端导航特别慢

WSL2 上 Gazebo RTF 常低于 1，加上 Nav2 计算量，卡顿正常。对策：`controller_frequency` 已降到 10Hz、DWB 采样数已减半；验收与日常练习优先用 lite 后端，Gazebo 留给性能较好的机器看物理观感。

9.5 发了 `task_command` 没反应

- 检查 `mission_state`: `detail=waiting_for_nav2` 说明 Nav2 action server 还没就绪，等激活完成。
- `detail=unknown_goal:xxx` 说明目标名不在 `named_goals` 表里，检查拼写。
- JSON 里引号要转义（外层单引号、内层 `\`），直接复制讲义命令最保险。

9.6 TF_OLD_DATA 刷屏

上次课的老问题：残留的 `gz/rviz` 进程还在发旧时钟。`bash scripts/cleanup_ros2.sh --all` 后重启。

10. 本节验收清单

- ☐ 能讲清第二节留下的问题：漂移是什么；`map → odom` 是矫正账本；为何不直接改 `/odom`。
- ☐ 能按主线四步说明导航：加载冻结地图 → 初始位姿 + AMCL → 行走中更新 `map→odom` → 规划跟线到目标。
- ☐ 能解释「怎么去」的四层：为什么规划要在代价地图而不是黑白图上做；路径为何不走直线；DWB 的「动态窗口」窗在哪里；卡住后 `spin/backup` 是谁调的。
- ☐ 知道为何要给初始位姿，以及定位发散时用 2D Pose Estimate 重设。
- ☐ `navigation.launch.py` 启动后定位收敛；RViz 单目标能到。
- ☐ 命名目标成功；运动中取消能停（若已做 LAB3）。
- ☐ `./scripts/test_cp3.sh` 输出 `MECAMIND_CP3_OK`。
- ☐ （加分）能指认 `map_server` / `amcl` / `planner` / `controller` / `bt_navigator` 在主线中各属哪一步，而不是只会背名单。

11. 课后作业

1. 在 `mecamind_named_goals.yaml` 里增加一个自己命名的点（用 RViz Publish Point 取坐标），导航过去。
2. 把 `mecamind_patrol_waypoints.yaml` 的 `loop_count` 改为 2，观察巡航两圈的 `mission_state` 变化。
3. 思考题：把 `robot_model_type` 改成 `DifferentialMotionModel` 会发生什么？（可以真的改了跑一次 CP3 对比。）
4. 进阶：用 `ros2 action send_goal` 直接发 `NavigateToPose`，观察它与 `task_command` 路径在 `rqt_graph` 上的区别。

12. 下一次课预告

第四节课接入真实感知：YOLO 目标检测 ROS 2 节点（图片/视频/Gazebo 相机三种输入），发布结构化检测消息。导航 + 感知在第五节课合流成"视觉跟随"。

本讲义中的截图（Gazebo 场景、RViz 启动/导航/巡航画面、节点话题图）与第 8 节的验收数据，均来自编写时在 WSL2 + Gazebo Harmonic 环境中的真实运行，非示意图。复现步骤即讲义正文命令。